REAL-TIME FRAUD DETECTION MICROSERVICES ARCHITECTURE

Category: Financial Technology | Fraud Detection | Microservices Architecture | Real-Time Analytics | Risk Management

Timeline: 15 Days | AI-Driven System Design & Architecture Project

Target: Undergraduate / MBA / Engineering (CS/IT) / Data & Analytics / FinTech Enthusiasts

TASK:

Design a microservices-based fraud detection platform that analyses transactions in real-time using rule engines, anomaly detection, and graph-based relationship analysis

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

ZeTheta ADD EXPERIENCE TO KNOWLEDGE

EXECUTIVE SUMMARY

Real-Time Fraud Detection Microservices Architecture is an advanced financial technology project focused on designing a scalable, production-grade fraud detection platform capable of analysing transactions in real time. The project requires participants to architect a distributed system using microservices principles, event-driven design, and advanced fraud detection techniques such as rule engines, anomaly detection, and graph-based relationship analysis.

The system processes high-volume transaction data from multiple payment channels-including card networks, digital payments, and UPI-while maintaining strict latency requirements and regulatory compliance. By decomposing a legacy monolithic system into independently deployable microservices, participants build a resilient architecture that supports real-time decision-making, fault isolation, and scalability.

The project integrates multiple detection layers: rule-based systems for known fraud patterns, machine learning models for anomaly detection, and graph analytics for uncovering complex fraud networks. These signals are combined through a risk scoring engine that produces explainable, audit-ready decisions aligned with regulatory frameworks such as RBI guidelines and PCI DSS.

Participants also design a gamified simulation, positioning themselves as platform architects responding to a surge in fraud incidents. This simulation introduces real-world constraints such as system failures, attack scenarios, and operational challenges, encouraging strategic thinking and problem-solving under pressure.

Overall, the project bridges financial domain knowledge with modern system architecture, enabling participants to understand how large-scale fintech platforms detect, prevent, and respond to fraud in real time while balancing accuracy, speed, and compliance.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART A: TECHNICAL TRAINING MATERIAL & DOMAIN KNOWLEDGE

This section provides the comprehensive technical foundation required for architecting a production-grade, real-time fraud detection microservices platform. Interns are expected to study every sub-section thoroughly before commencing design or implementation. The material covers microservices architecture patterns, event-driven design, fraud detection algorithms, service mesh concepts, DevOps strategy, and regulatory compliance frameworks. All content is presented with a global perspective, with India as the prime sample market.

SECTION A1: MICROSERVICES ARCHITECTURE FUNDAMENTALS

A1.1 Monolith VS Microservices: Architectural Evolution

A monolithic application is a single deployable unit in which all business logic, data access, and user-interface code reside within one process. While monoliths are simple to develop, test, and deploy at small scale, they become unwieldy as organisations grow. Tight coupling means a change to one module necessitates redeploying the entire application, increasing risk and slowing release cadence. Scaling is limited to vertical scaling (larger servers) because the entire application must be replicated even when only one component is under load. Fault isolation is poor: a memory leak in the reporting module can bring down the payments path. These drawbacks motivate the transition to microservices.

Microservices architecture structures an application as a collection of loosely coupled, independently deployable services, each owning a bounded context from domain-driven design. Each service encapsulates a specific business capability - for instance, Transaction Ingestion, Rule Engine, Anomaly Detection, Graph Analysis, Risk Scoring, Case Management, Notification, and Audit. Services communicate via well-defined APIs (synchronous REST or gRPC) or asynchronous message brokers (Apache Kafka, RabbitMQ). Each service can be developed, deployed, and scaled independently. This architecture enables polyglot persistence (different databases per service) and polyglot programming (different languages per service where appropriate).

Key Characteristics Of Microservices: Single Responsibility Principle applied at the service level; independent deployment with zero-downtime releases; decentralised data management with each service owning its datastore; design for failure using circuit breakers, bulkheads, and retries; infrastructure automation via CI/CD pipelines, containerisation (Docker), and orchestration (Kubernetes); observability through distributed tracing (Jaeger, Zipkin), structured logging (ELK stack), and metrics (Prometheus/Grafana).

In the context of a fraud detection platform, microservices architecture is essential because different detection techniques have vastly different computational profiles. A rule engine requires low-latency, deterministic evaluation; an anomaly detection model may require GPU acceleration; a graph analysis engine needs access to a specialised graph database. Separating these into individual services allows each to be optimised, scaled, and updated independently without affecting the others.

The transition from monolith to microservices is not a binary switch but a gradual migration. The Strangler Fig pattern is the recommended approach: new features are built as microservices while the monolith is gradually decomposed. An anti-corruption layer (API adapter) sits between the legacy monolith and the new services, translating between the legacy data model and the new domain models. This allows the team to retire monolith modules one at a time while maintaining system stability throughout the migration.

A1.2 Domain-Driven Design And Bounded Contexts

Domain-Driven Design (DDD) is the foundational methodology for identifying service boundaries. In DDD, the problem domain is broken into bounded contexts, each representing a cohesive area of business logic with its own ubiquitous language. For a fraud detection platform, the following bounded contexts emerge naturally:

- Transaction Ingestion: Receiving and normalising transaction data from multiple payment channels (POS terminals, e-commerce gateways, mobile wallets, ATM networks, UPI systems). This context handles protocol translation, data enrichment with device fingerprinting and IP geolocation, and publishing normalised transaction events to the event backbone.

- Fraud Analysis: The core detection engines including the Rule Engine (deterministic, configurable rule evaluation), Anomaly Detection (ML-based scoring using supervised and unsupervised models), and Graph Analysis (relationship-based pattern detection using a property graph database). Each of these may be one or more microservices depending on scaling requirements.

- Risk Scoring: Aggregating signals from the Rule Engine, Anomaly Detection, and Graph Analysis into a composite risk score with configurable weighting. Producing explainable decision outputs that specify which signals contributed to the final score and by how much. Applying configurable thresholds to determine the action (approve, step-up, review, decline).

- Case Management: Human-in-the-loop workflow for investigating flagged transactions. Assignment logic (round-robin, skill-based routing, priority queuing), investigation tools (transaction timeline, customer communication history, related cases), and decision workflow (confirm fraud, false positive, escalate). Integration with chargeback systems.

- Notification: Multi-channel alerting to cardholders (SMS, email, push notification, in-app alert), merchants (webhook, email), and internal teams (Slack, PagerDuty). Template management with localisation support (Hindi, English, Marathi, Tamil, etc. for Indian market). Delivery tracking and retry logic.

- Audit and Compliance: Maintaining immutable audit trails for every fraud decision, rule change, model deployment, threshold modification, and user action. Regulatory reporting (RBI fraud reporting, PCI DSS audit support, SAR filing for FATF compliance). Data retention management compliant with RBI (7 years) and GDPR (right to erasure) requirements.

- Customer Profile: Managing customer behavioural profiles built from historical transaction patterns. Features include typical transaction amounts, preferred merchants, usual transaction times, home and work geolocations, device inventory, and communication preferences. The profile is updated incrementally as new transactions are processed.
- Reference Data: Maintaining merchant category codes (MCCs), Bank Identification Number (BIN) databases, geolocation mappings, country risk scores, currency exchange rates, holiday calendars (for different markets), and watchlists (sanctioned entities, known fraud accounts, compromised card ranges).

Each bounded context maps to one or more microservices. Communication between bounded contexts is through published interfaces (APIs or events), never through shared databases. This ensures each context can evolve independently. An anti-corruption layer is used when integrating with legacy systems to prevent legacy data models from leaking into new services.

Context Mapping Patterns: The relationships between bounded contexts can be modelled as Customer-Supplier (Transaction Ingestion supplies data to Fraud Analysis), Conformist (Notification conforms to the event schema published by Risk Scoring), Anti-Corruption Layer (integrating with the legacy monolith), or Shared Kernel (shared data types like transaction IDs or risk enumerations). Interns must produce a Context Map diagram showing all bounded contexts and their relationships as part of their C4 Model deliverables.

A1.3 The C4 Model: Architectural Documentation

The C4 model is a hierarchical approach to documenting software architecture, developed by Simon Brown. It consists of four levels: Context (Level 1) shows the system in scope and its relationships with users and other systems; Container (Level 2) zooms into the system to show high-level technical building blocks (applications, data stores, message brokers); Component (Level 3) decomposes each container into its components; and Code (Level 4) maps components to code-level structures such as classes and interfaces. For this project, interns must produce Levels 1 through 3 as primary deliverables. Level 4 is optional but encouraged for the Rule Engine service.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

At Level 1 (System Context), the fraud detection platform interacts with external systems including the Card Payment Network (Visa, Mastercard, RuPay for India), the Core Banking System (which provides real-time balance information and account status), the Customer Mobile Application (which receives notifications and enables step-up authentication), Third-Party Enrichment Services (device fingerprinting providers such as ThreatMetrix, IP geolocation services such as MaxMind, and velocity data providers), Regulatory Reporting Systems (RBI for India, FinCEN for USA, FCA for UK), and the Legacy Monolith System (during the migration period, via the anti-corruption layer). Each external system must be shown with its communication protocol, data flow direction, and integration pattern.

At Level 2 (Container Diagram), the platform is decomposed into containers. A container in C4 parlance is a separately deployable unit: a microservice, a database, a message broker, a reverse proxy, or a front-end application. The intern must show all 8-12 microservices, their databases

(PostgreSQL for transactional data, Redis for caching and feature store, Apache Cassandra or TimescaleDB for time-series analytics, Neo4j for graph analysis, Elasticsearch for log storage and full-text search), the message broker (Apache Kafka with specific topic designs), the API gateway (Kong or AWS API Gateway), the service mesh sidecar proxies (Istio/Envoy), the monitoring stack (Prometheus, Grafana, Jaeger, ELK/EFK), and the front-end applications (Case Management Web UI, Analytics Dashboard, Rule Management Console).

At Level 3 (Component Diagram), each container is decomposed into its internal components. For the Rule Engine service, this would include: the Rule Parser (reads YAML/JSON rule definitions), the Condition Evaluator (evaluates rule conditions against transaction data), the Temporal Aggregator (computes windowed aggregations like transaction counts over time), the Action Executor (executes rule actions like flagging or score adjustment), the Rule Cache (in-memory cache of compiled rules for low-latency evaluation), and the Rule Management API (CRUD operations for rules with versioning). Interns must create Level 3 diagrams for at least four core services.

A1.4 Inter-Service Communication Patterns

Communication between microservices in a fraud detection platform must balance latency requirements against reliability and decoupling. Two fundamental patterns exist: synchronous (request-response) and asynchronous (event-driven). In practice, a fraud detection system uses both, with the choice depending on whether the caller needs an immediate response.

Synchronous Communication (gRPC): gRPC is preferred over REST for internal service-to-service calls in a fraud detection platform because of its binary serialisation (Protocol Buffers), bidirectional streaming, HTTP/2 multiplexing, and strongly-typed contracts. REST (HTTP/JSON) is used for external-facing APIs (the API gateway exposes REST to external consumers) because of ubiquity and tooling. For the critical path - from transaction ingestion to risk decision - gRPC calls between the Rule Engine, Anomaly Detection, and Risk Scoring services must complete within a combined latency budget of 100 milliseconds at the 99th percentile. Interns must specify latency SLAs per service and per hop.

Synchronous Communication (REST): REST is used for external-facing APIs exposed through the API gateway. These include the transaction submission endpoint (POST /api/v1/transactions), the risk score inquiry endpoint (GET /api/v1/transactions/{id}/risk-score), the case management endpoints (CRUD operations on fraud investigation cases), the rule management endpoints (CRUD operations on fraud detection rules), and the dashboard data endpoints (aggregated statistics for the monitoring dashboards). All REST APIs must follow OpenAPI 3.0 specification with versioned paths (/api/v1/, /api/v2/) and consistent error response formats.

Asynchronous Communication (Apache Kafka): Apache Kafka serves as the backbone for event-driven communication. Key topics include: transactions.raw (raw transaction events from payment channels, partitioned by card_number_hash for ordering), transactions.enriched (transactions enriched with device, geo, and customer profile data), fraud.alerts (flagged transactions with risk scores above threshold), fraud.decisions (final accept/reject/review decisions after human review or auto-decision), audit.events (immutable audit log of all system

actions), notifications.outbound (notification requests for email, SMS, push), and graph.updates (entity and relationship updates for the graph database). Kafka's partitioning strategy is critical: transactions should be partitioned by card_number_hash to ensure all transactions for a given card are processed in order by the same consumer instance, enabling stateful pattern detection.

Interns must design an Event Storming diagram showing all domain events, commands, aggregates, and read models. The diagram must cover the entire transaction lifecycle from ingestion to decision to notification to case management. The Event Storming exercise should produce at least 50 domain events with clear aggregate boundaries and policy definitions.

A1.5 Data Consistency Patterns

In a microservices architecture where each service owns its data, maintaining consistency across services is a fundamental challenge. Distributed transactions (two-phase commit) are generally avoided in microservices due to their tight coupling, latency overhead, and single point of failure (the coordinator). Instead, the following patterns are used:

The Saga Pattern: A saga is a sequence of local transactions where each local transaction updates its service's database and publishes an event to trigger the next step. If a step fails, compensating transactions are executed to undo the preceding steps. For example, a Fraud Investigation Saga might involve: (1) Card Management Service freezes the card (compensating action: unfreeze), (2) Case Management Service creates an investigation case (compensating action: cancel case), (3) Notification Service sends alert to cardholder (compensating action: send correction notification), (4) Audit Service logs the investigation initiation (no compensation needed - audit logs are append-only). Two implementation patterns exist: orchestration (a central Saga Orchestrator directs participants) and choreography (each service reacts to events independently). Orchestration is preferred for complex fraud investigation workflows; choreography for simpler enrichment pipelines.

CQRS (Command Query Responsibility Segregation): CQRS separates write operations (commands) from read operations (queries) into different models. In the fraud detection platform, the write side processes transaction events through the detection pipeline with strong consistency guarantees, while the read side serves analytics dashboards and reporting interfaces from eventually consistent materialised views. Materialised views are updated by Kafka Streams or ksqlDB consumers that read from the fraud.decisions topic and maintain pre-aggregated statistics (e.g., fraud rate by hour, by merchant category, by geography). This separation ensures that heavy analytics queries do not impact the latency-sensitive transaction processing pipeline.

Event Sourcing (Optional): Event sourcing stores the state of an entity as a sequence of events rather than the current state. For the Audit Service, event sourcing is a natural fit because every state change is preserved as an immutable event. For other services, event sourcing adds complexity and is optional. However, the transaction pipeline already uses event-driven architecture, which provides many of the benefits of event sourcing (auditability, replayability) without the full complexity.

Eventual Consistency: The fraud detection platform embraces eventual consistency for non-critical paths (dashboard analytics may lag by a few seconds, customer profile updates may take a few seconds to propagate) while enforcing strong consistency for the critical path (the

fraud decision for a given transaction must be deterministic and based on the latest available data). The system must clearly document which operations are strongly consistent and which are eventually consistent, with maximum staleness guarantees for the latter.

SECTION A2: FRAUD DETECTION DOMAIN KNOWLEDGE

A2.1 Card-Present VS Card-Not-Present Fraud

Card-Present (CP) fraud occurs when a physical card or its clone is used at a point-of-sale (POS) terminal. Common CP fraud types include: counterfeit card fraud (magnetic stripe cloning via skimming devices installed on ATMs or POS terminals), lost/stolen card fraud (physical theft of the card followed by rapid spending before the cardholder reports the loss), and card-never-received fraud (interception of newly issued cards during postal delivery). EMV chip technology has significantly reduced counterfeit fraud in markets with high chip adoption, but magnetic stripe fallback transactions remain vulnerable in some regions, and contactless payment fraud (tap-to-pay without PIN for low-value transactions) is an emerging concern.

Card-Not-Present (CNP) fraud occurs in transactions where the physical card is not presented - primarily e-commerce, mobile commerce, mail order, and telephone order (MOTO) transactions. CNP fraud has grown dramatically as commerce has shifted online, and now accounts for approximately 75% of all card fraud globally. Common CNP fraud types include: account takeover (ATO) where stolen credentials are used to make purchases or change account details, enumeration attacks (automated testing of large volumes of stolen card numbers with small-value transactions to identify valid cards), friendly fraud (legitimate cardholders disputing valid transactions to obtain refunds, also known as first-party fraud or chargeback fraud), triangulation fraud (fraudsters sell goods at discounted prices, purchase them with stolen cards, and ship directly to buyers), and synthetic identity fraud (creating fictitious identities by combining real and fabricated personal information to open new accounts).

India-Specific Context: The Reserve Bank of India (RBI) has implemented several measures that significantly affect fraud detection system design. Tokenisation of card-on-file data (effective October 2022) requires merchants to replace stored card numbers with tokens, impacting fraud detection models that previously used raw card numbers as features. Additional Factor of Authentication (AFA) for online card transactions - typically OTP via SMS - adds a layer of protection against CNP fraud but introduces new attack vectors such as SIM-swap fraud and OTP interception via malware on the cardholder's device. UPI (Unified Payments Interface) transactions, unique to India and processing over 10 billion transactions per month, present their own fraud patterns including social engineering via fake UPI collect requests, malicious QR codes that redirect payments, impersonation of customer support agents, and UPI PIN phishing through spoofed apps. RuPay, India's domestic card network, has its own BIN ranges and processing rules that the system must accommodate alongside Visa and Mastercard.

A2.2 Rule-Based Fraud Detection

Rule engines form the first line of defence in real-time fraud detection. A rule is a conditional statement that evaluates transaction attributes against predefined thresholds or patterns. Rules are

deterministic, interpretable, and fast to evaluate, making them suitable for known fraud patterns and regulatory requirements.

Rule Categories and Examples:

- Velocity Rules: More than N transactions in T minutes (e.g., more than 5 transactions in 10 minutes from the same card). More than N distinct merchants in T minutes. Total transaction amount exceeding threshold within a time window.
- Amount Rules: Single transaction exceeding a threshold (e.g., 500,000 INR for domestic, 100,000 INR for international). Transaction amount significantly deviating from cardholder's average (e.g., more than 5 standard deviations above the 90-day mean).
- Geographic Rules: Transactions in two different countries within a physically impossible travel time (e.g., India and USA within 2 hours). Transaction from a country not in the cardholder's travel history. Transaction from a high-risk country (based on the Reference Data service's country risk scores).
- Merchant Category Rules: First-time transaction at a high-risk MCC such as gambling (MCC 7995), wire transfer (MCC 4829), cryptocurrency (MCC 6051), or quasi-cash (MCC 6050). Sudden change in merchant category spending pattern.
- Watchlist Rules: Card number on a known compromised card list (e.g., shared by Visa/Mastercard after a merchant breach). Merchant on a fraud watchlist. IP address on a known botnet or proxy list.
- Behavioural Rules: Transaction at an unusual time of day for this cardholder. Transaction in a category the cardholder has never used before. Sudden increase in online transaction frequency for a cardholder who primarily uses the card at POS terminals.

A configurable rule engine must support the following capabilities: rule authoring (business users define rules through a management UI without code changes), rule versioning (each rule change creates a new version; old versions are archived with full audit trail), rule prioritisation (rules execute in priority order with short-circuit evaluation to minimise latency), rule chaining (the output of one rule feeds into another, enabling complex multi-step evaluation), rule simulation (test new rules against 30 days of historical data before deploying to production, measuring detection rate and false positive rate), rule A/B testing (run a new rule in shadow mode alongside the existing rule and compare outcomes before full activation), and rule performance monitoring (measure false positive rate, detection rate, evaluation latency, and trigger frequency per rule, with automated alerts when metrics degrade).

Rule Configuration Framework: Interns must design a YAML/JSON schema for rule definitions that supports conditions (field, operator, value, temporal_window), logical operators (AND, OR, NOT with unlimited nesting depth), entity scopes (cardholder, merchant, terminal, network, device), actions (flag, block, score_adjust, notify, step_up_auth), metadata (rule_id, version, author, effective_date, expiry_date, severity, description), and performance constraints (maximum evaluation time in milliseconds). The schema must be extensible to accommodate

new rule types without code changes. A JSON Schema or YAML schema definition must be provided for validation.

A2.3 Anomaly Detection And Machine Laarnign

While rules detect known fraud patterns, machine learning models detect unknown or evolving patterns through anomaly detection. The ML-based detection system operates in parallel with the rule engine, providing complementary coverage.

Supervised Models: Trained on labelled historical data (fraud/not-fraud) and include: logistic regression (fast, interpretable baseline with feature coefficients that serve as explainability), gradient-boosted trees (XGBoost, LightGBM - state-of-the-art accuracy for tabular data with built-in feature importance), random forests (ensemble method providing robust predictions with lower variance than single decision trees), and neural networks (LSTM or Transformer architectures for sequence modelling of transaction histories, capturing temporal patterns that tree-based models miss).

Unsupervised Models: Detect anomalies without labelled data and include: isolation forests (efficient anomaly detection that isolates anomalies by random partitioning - anomalies require fewer partitions), autoencoders (neural networks trained to reconstruct normal transaction patterns; transactions with high reconstruction error are flagged as anomalous), variational autoencoders (generative model that learns a latent representation of normal behaviour), one-class SVM (defines a boundary around normal data points; points outside the boundary are anomalies), and clustering-based methods (DBSCAN for identifying outlier transaction clusters, k-means for defining normal behavioural clusters).

Feature Engineering: Feature engineering is the most critical aspect of fraud ML. Features fall into categories: (1) Transaction-level: amount, MCC, channel (POS/ecom/ATM/UPI), currency, time_of_day, day_of_week, is_weekend, is_holiday; (2) Cardholder-level: average transaction amount over 30/60/90 days, transaction frequency, preferred merchants (top 10), typical transaction hours, home geolocation, card age; (3) Velocity features: number of transactions in last 1/5/10/30/60 minutes, cumulative amount in last 1/6/12/24 hours, number of distinct merchants in last hour, number of distinct countries in last 24 hours; (4) Distance features: geographic distance from last transaction, distance from home location, distance from card-issuing bank, distance from nearest previously visited merchant; (5) Device/network features: device fingerprint match score, IP geolocation, proxy/VPN detection, browser characteristics, mobile device IMEI, whether the device is jailbroken/rooted; (6) Derived features: ratio of current amount to historical average, z-score of amount relative to cardholder history, time since last transaction, recency of first transaction with this merchant.

Model Serving Architecture: For real-time inference, the ML model must be served with sub-10ms latency per prediction. Options include TensorFlow Serving (supports TensorFlow and Keras models with gRPC and REST APIs), ONNX Runtime (cross-framework model serving with optimised inference), Triton Inference Server (NVIDIA's multi-framework server with dynamic batching and GPU scheduling), or a custom serving layer using Python (FastAPI with pre-loaded models in memory). The model service must support A/B testing (serving multiple model versions simultaneously with configurable traffic splits), model warm-up (pre-loading

models into memory on container start to eliminate cold-start latency), feature store integration (reading pre-computed features from Redis or Feast with sub-millisecond latency), model monitoring (detecting data drift using Population Stability Index and prediction drift using KS test), and model ensemble (combining predictions from multiple models using stacking, weighted average, or maximum).

A2.4 Graph-Based Relationship Analysis

Graph analysis is the most sophisticated detection technique and is essential for identifying organised fraud rings that evade rule-based and ML-based detection. A fraud graph connects entities - cards, accounts, devices, IP addresses, phone numbers, email addresses, physical addresses, and merchants - through relationships such as transacted_with, shared_device, shared_address, shared_phone, same_beneficiary, and linked_via_transaction. Fraud rings often share infrastructure: multiple synthetic identities may use the same device, ship to the same address, operate from the same IP subnet, or share phone numbers for OTP receipt.

Graph Database Technology: Neo4j is recommended for the fraud graph due to its native graph storage (index-free adjacency for O(1) relationship traversal), mature Cypher query language, graph data science library (community detection, centrality, path finding), and ACID transaction support. Amazon Neptune (if deploying on AWS) and JanusGraph (open-source, scalable on Cassandra/HBase backend) are alternatives. The choice must be justified based on query complexity, scalability requirements, and operational considerations.

Graph Query Patterns For Fraud Detection:

- Community Detection: The Louvain algorithm identifies clusters of tightly connected entities. In a fraud context, a community of cards, devices, and addresses that are densely connected but weakly connected to the rest of the graph may indicate a fraud ring. The intern must specify the modularity threshold for flagging a community as suspicious.
- Centrality Analysis: PageRank identifies the most influential nodes in the graph. In a fraud context, a device or IP address with high PageRank that connects many otherwise unrelated accounts may be a shared fraud infrastructure node. Betweenness centrality identifies bridge nodes that connect different communities - these may be money mule accounts that facilitate fund movement between fraud rings.
- Path Analysis: Shortest path queries determine the degree of separation between a flagged entity and known fraud accounts. A card that is only 2 hops away from a confirmed fraud account (e.g., they shared a device with an intermediary account that was involved in fraud) has higher risk than a card that is 6 hops away.
- Temporal Graph Analysis: Identifying rapid growth in connections to new entities, which may indicate account farming (creating many accounts in quick succession). The intern must design a time-windowed query that detects entities with a connection growth rate exceeding a threshold (e.g., more than 10 new connections in 7 days).

- Subgraph Matching: Matching known fraud topologies (templates) against the live graph. For example, a triangulation fraud template might be: Card A transacts with Merchant M, Merchant M ships to Address X, Address X also receives shipments from Merchant N paid by Card B, and Cards A and B share a Device D. The intern must define at least 3 fraud topology templates.

Graph Update Strategy: The fraud graph must be updated in near-real-time as new transactions arrive. Two patterns exist: synchronous update (the graph is updated as part of the transaction processing pipeline, adding 5-15ms latency but ensuring the graph is always current) and asynchronous update (graph updates are streamed via Kafka and applied with a small delay of 1-5 seconds, reducing transaction latency but introducing a window where the graph is stale). Interns must evaluate both approaches, specify the chosen strategy with quantitative justification (latency impact vs staleness window), and design a reconciliation mechanism if asynchronous update is chosen.

A2.5 Risk Scoring And Explainability

The Risk Scoring service aggregates signals from the Rule Engine, Anomaly Detection, and Graph Analysis services into a composite risk score between 0 (no risk) and 1000 (maximum risk). The aggregation function must be configurable and support multiple strategies: weighted linear combination (Score = w1*RuleScore + w2*MLScore + w3*GraphScore, where weights are configurable per channel and market), maximum-of-signals (Score = max(RuleScore, MLScore, GraphScore) - conservative, flags if any engine is concerned), and Bayesian combination (treating each engine's output as independent evidence and combining using Bayes' theorem).

Configurable thresholds determine the action based on the composite risk score. The thresholds must be configurable per market, per channel, and per card product:

Score Range	Action	SLA	Description
0 - 199	Auto-Approve	< 100ms	Transaction is processed normally
200 - 599	Step-Up Authentication	< 5s	Request OTP, biometric, or challenge question
600 - 799	Manual Review Queue	< 4 hours	Route to fraud analyst for investigation
800 - 1000	Auto-Decline	< 100ms	Transaction is blocked; alert sent to cardholder

Explainability: Regulatory compliance requires that every fraud decision be explainable. The Risk Scoring service must produce a human-readable explanation for each decision, specifying: which rules triggered (rule_id, rule_name, matched condition values), which ML features contributed most to the anomaly score (using SHAP values for tree-based models or LIME for

model-agnostic explanations, reporting the top 5 contributing features), which graph relationships were suspicious (path to known fraud, community suspicion score, centrality rank), and the weight each engine contributed to the final score. This explanation must be stored in the Audit service and be retrievable for regulatory examination and customer dispute resolution.

Regulatory Frameworks: PCI DSS (Payment Card Industry Data Security Standard) mandates encryption of cardholder data at rest and in transit, network segmentation, access controls, vulnerability management, and audit logging. RBI guidelines for India require reporting of fraud cases above specified thresholds (2 lakh INR for card fraud), maintaining fraud monitoring systems with real-time capability, implementing additional factor authentication, and filing Suspicious Transaction Reports (STRs) with the Financial Intelligence Unit (FIU-IND). GDPR (for European cardholders) requires data minimisation, purpose limitation, right to erasure (which conflicts with RBI's 7-year retention requirement - the system must handle this conflict by jurisdiction), and lawful basis for processing (legitimate interest for fraud prevention). The system architecture must be designed to comply with all three frameworks simultaneously, with jurisdiction-based configuration to handle conflicting requirements.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A3: SERVICE MESH, API GATEWAY, AND OBSERVABILITY

A3.1 Service Mesh Design With ISTIO/ENVOY

A service mesh provides a dedicated infrastructure layer for handling service-to-service communication, decoupling networking concerns from application code. Istio, built on top of Envoy sidecar proxies, provides: mutual TLS (mTLS) for zero-trust communication between all services (no service trusts another by default; every request is authenticated and encrypted), traffic management (canary deployments with configurable traffic percentages, circuit breaking with configurable error thresholds and recovery windows, rate limiting per service, retries with exponential backoff and jitter, and request timeouts), and observability (automatic metrics collection for request count, latency, and error rate per service pair; distributed tracing injection via OpenTelemetry; and access logging for every request).

In a fraud detection platform where data sensitivity is paramount, mTLS ensures that even internal communications between the Transaction Ingestion service and the Rule Engine are encrypted and mutually authenticated. This prevents lateral movement by an attacker who compromises a single service - they cannot impersonate another service to access cardholder data.

Interns must design: destination rules (specifying connection pool sizes, outlier detection thresholds, and TLS settings per service), virtual services (defining traffic routing for canary and blue-green deployments with percentage-based traffic splitting), authorisation policies (restricting which services can communicate with which, following the principle of least privilege - e.g., the Notification service should not be able to call the Rule Engine directly), and peer authentication policies (enforcing strict mTLS for all inter-service communication with no exceptions).

A3.2 API Gateway Design

The API gateway (Kong, AWS API Gateway, or Envoy as edge proxy) serves as the single entry point for all external traffic. It handles: authentication (JWT validation with RS256, OAuth 2.0 client credentials flow for machine-to-machine communication, API key validation for legacy integrations), authorisation (RBAC - role-based access control mapping JWT claims to permissions), rate limiting (per-client, per-endpoint, per-IP, and global limits), request transformation (mapping external REST APIs to internal gRPC calls using a transcoding layer), response caching (cache risk score lookups for 5 seconds to handle duplicate requests from retry-happy clients), SSL/TLS termination (with TLS 1.3 for external connections), request validation (schema validation against OpenAPI spec before forwarding to backend), and logging (access logs for every request with correlation ID for tracing).

Rate Limiting Design: Rate limiting is critical to prevent enumeration attacks (fraudsters testing large volumes of stolen card numbers through the transaction endpoint). The gateway must implement tiered rate limits: per-merchant limits (based on contracted throughput, e.g., 1000 TPS for large acquirers, 100 TPS for small merchants), per-IP limits (to detect distributed attacks, e.g., 50 requests per minute per IP), per-endpoint limits (different limits for transaction

submission vs read operations), global limits (to protect backend services from overload, e.g., total system capacity of 50,000 TPS), and adaptive rate limiting (dynamically tightening limits when the system detects attack patterns - e.g., if the fraud rate spikes above 5%, reduce the per-merchant limit by 50%). Rate limit counters must use a distributed store (Redis with atomic increment) to work correctly across multiple gateway instances. Response headers (X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset) must be included so clients can self-regulate.

A3.3 Observability: Monitoring, Logging and Tracing

Observability in a distributed fraud detection system is non-negotiable. The three pillars are: metrics (Prometheus for collection with scrape intervals per service, Grafana for visualisation with pre-built dashboards), logs (structured JSON logging with mandatory fields, shipped via Fluentd/Fluent Bit to Elasticsearch, visualised in Kibana), and traces (distributed tracing with Jaeger or Zipkin using OpenTelemetry SDK instrumentation, with trace context propagation across both synchronous gRPC calls and asynchronous Kafka messages). Every transaction must carry a correlation ID (trace_id) from ingestion through to decision, enabling end-to-end latency analysis and debugging.

Key Metrics and SLAs: Transaction processing latency: p50 < 50ms, p95 < 150ms, p99 < 200ms for the entire pipeline. Fraud detection rate: > 95% of known fraud correctly flagged (measured by monthly reconciliation against confirmed fraud cases). False positive rate: < 0.5% of legitimate transactions incorrectly flagged (measured by analyst decision outcomes). System availability: 99.99% for the transaction processing path (critical), 99.9% for case management and dashboards (non-critical). Kafka consumer lag: < 1000 messages per partition at steady state. Error rate per service: < 0.1% of requests resulting in 5xx errors. Model inference latency: p99 < 10ms. Graph query latency: p99 < 50ms.

Alerting Strategy: Alerting must be configured with severity levels and escalation paths: P1 (critical - PagerDuty page, auto-escalation after 5 minutes: transaction pipeline completely down, fraud detection rate drops below 80%, data breach indicators), P2 (high - Slack #alerts channel, escalation after 30 minutes: latency SLA breached for more than 5 minutes, Kafka consumer lag exceeding 10,000 messages, single service error rate above 1%), P3 (medium - daily digest email: false positive rate increased by 20% over 7-day baseline, model drift detected, non-critical service degradation), P4 (low - weekly review dashboard: infrastructure costs exceeding budget, performance optimization opportunities identified). Each P1 and P2 alert must have an associated runbook specifying symptoms, diagnosis steps, remediation steps, and escalation procedures.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A4: DEVOPS, DEPLOYMENT, AND DISASTER RECOVERY

A4.1 Containerisation and Orchestration

Every microservice must be containerised using Docker with multi-stage builds to minimise image size (production images should be under 100MB). Base images should be distroless or Alpine-based to reduce attack surface (fewer installed packages = fewer vulnerabilities). Each Dockerfile must include: a multi-stage build (builder stage for compilation, runtime stage for execution), a non-root user directive (USER 1000), a health check endpoint (/healthz), explicit EXPOSE directives for service ports, and LABEL directives for image metadata (version, commit hash, build date).

Kubernetes is the orchestration platform, with separate namespaces for development, staging, and production environments. Resource requests and limits must be specified for every container to enable proper scheduling and prevent noisy-neighbour effects. Kubernetes manifests (or Helm charts) must define: Deployments (with rolling update strategy specifying maxSurge and maxUnavailable), Services (ClusterIP for internal, LoadBalancer for external), ConfigMaps (environment-specific configuration such as feature flags, thresholds, endpoints) and Secrets (database credentials, API keys - sourced from HashiCorp Vault via external-secrets operator), HorizontalPodAutoscalers (scaling rules based on CPU utilisation, memory utilisation, or custom metrics such as Kafka consumer lag), PodDisruptionBudgets (ensuring minimum availability during node maintenance - e.g., minAvailable: 2 for the Rule Engine), and NetworkPolicies (restricting inter-service communication to only permitted paths, matching the service communication matrix designed in the security section).

A4.2 Blue-Green and Canary Deployment Strategies

Blue-green deployment maintains two identical production environments (blue and green). The active environment (e.g., blue) serves all traffic while the new version is deployed to the inactive environment (green). After smoke testing (automated health checks, integration test suite, manual spot checks), traffic is switched from blue to green via DNS or load balancer reconfiguration. If issues are detected post-switch, traffic is immediately switched back to blue. This provides zero-downtime deployments with instant rollback capability at the cost of requiring double the infrastructure during the deployment window.

Canary deployment gradually shifts traffic from the old version to the new version, starting with a small percentage (e.g., 5%) and increasing as confidence grows (5% → 25% → 50% → 100% over a period of hours or days). Automated canary analysis (using tools like Kayenta or custom Prometheus queries) compares error rates, latencies, and business metrics (fraud detection rate, false positive rate) between the old and new versions. If the canary shows degradation beyond a configurable threshold, traffic is automatically rolled back to 100% old version. Canary deployments are more resource-efficient than blue-green but require sophisticated traffic management (Istio virtual services with weighted routing) and automated metric comparison (which must be carefully calibrated to avoid both false rollbacks and missed regressions).

Deployment Strategy Per Service: Interns must design a differentiated deployment strategy: Canary deployment for the ML Model Service (to validate model accuracy on live traffic - even a 5% canary serving 5% of traffic generates statistically significant fraud detection metrics within a few hours), Blue-green deployment for the Rule Engine Service (to ensure rule changes are atomic and fully testable before traffic switch - rules either all apply or none apply, no partial rule set), Rolling updates for the Transaction Ingestion Service (with readiness probes ensuring zero message loss during deployments - old pods continue processing until new pods are ready), and Blue-green deployment for database schema migrations (using expand-contract pattern: first expand the schema to support both old and new versions, then migrate code, then contract the schema to remove deprecated columns).

A4.3 Disaster Recovery and Business Continuity

The disaster recovery (DR) plan must specify Recovery Point Objective (RPO) and Recovery Time Objective (RTO) for each service tier. Tier 1 (critical path: Transaction Ingestion, Rule Engine, Risk Scoring) requires RPO < 1 minute (synchronous database replication) and RTO < 5 minutes (automated failover with pre-provisioned standby). Tier 2 (supporting services: Case Management, Notification, Analytics) requires RPO < 15 minutes (asynchronous replication) and RTO < 30 minutes (semi-automated failover). Tier 3 (non-critical: Reporting, Admin tools) requires RPO < 1 hour and RTO < 4 hours.

The deployment topology must include: multi-region Kubernetes clusters (primary in Mumbai for Indian market, secondary in Singapore for Southeast Asia and disaster recovery), database replication strategy (synchronous replication within the primary region for strong consistency, asynchronous cross-region replication for DR), Kafka MirrorMaker 2 for cross-region topic replication (with offset translation for consumer failover), DNS-based failover with health checks (Route 53 failover routing or equivalent), and a detailed failover runbook specifying: detection criteria (automated health check failures), decision matrix (who authorises failover), execution steps (automated and manual), verification steps (confirming all services are operational in DR region), and rollback procedure (returning to primary when the issue is resolved).

Chaos Engineering: The architecture should include a chaos engineering strategy to proactively test resilience. Recommended experiments include: pod termination (randomly killing pods to test horizontal pod autoscaler and service availability), network partition (simulating network split between services to test circuit breaker and fallback behaviour), latency injection (adding artificial latency to downstream services to test timeout and retry configuration), Kafka broker failure (killing a Kafka broker to test partition leader election and consumer rebalancing), database failover (triggering a database failover to test connection retry and read-replica promotion). Interns should specify at least 5 chaos experiments with expected behaviour, actual behaviour validation criteria, and the monitoring dashboards that would detect the injected failure.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

This section defines the gamified simulation scenario that frames the entire project. The intern assumes the role of Platform Architect at ShieldPay Financial Services, a credit card company facing a 340% increase in fraud attempts, tasked with decomposing a monolithic legacy system into a modern microservices architecture.

SECTION B1: SIMULATION NARRATIVE AND SCENARIO

B1.1 The Legacy Monolith: Starting State

ShieldPay Financial Services is a mid-tier credit card issuer processing 2.4 million transactions per day across India, Southeast Asia, and the Middle East. Their fraud detection system is a Java monolith deployed on bare-metal servers in a co-located data centre in Mumbai. The monolith was originally built in 2012 and has accumulated 1.2 million lines of Java code over 12 years, with fraud detection rules hard-coded in Java switch statements (over 500 rules with no external configuration), a single Oracle 19c database for all data (transactions, customer profiles, fraud cases, audit logs, reporting - total size: 45 TB with 3-second query times for analytical queries), batch-mode ML model scoring that runs every 15 minutes via a cron job (processing the last 15 minutes of transactions against a logistic regression model trained 6 months ago), no service mesh or containerisation (services run as WAR files on Apache Tomcat 8.5), manual deployments that require a 4-hour maintenance window every Saturday night, and a single point of failure in the form of the Oracle database (no read replicas, no cross-region replication).

B1.2 The Crisis: What Triggered The Re-Architecture

In Q3 2025, ShieldPay experienced a 340% increase in fraud attempts driven by three concurrent threats:

- AI-Generated Synthetic Identities: A sophisticated fraud ring was creating fictitious customer identities using generative AI tools. These synthetic identities had realistic names, addresses (matching actual apartment buildings), Aadhaar numbers (algorithmically generated to pass checksum validation but not linked to real people), and even AI-generated photos for video KYC. The ring opened accounts, built spending history over 90 days, then executed coordinated fraud across hundreds of synthetic accounts simultaneously.

- Third-Party Data Breach: A major payment aggregator used by ShieldPay merchants suffered a data breach, exposing 12 million card numbers, expiry dates, and CVVs. The compromised data was sold on dark web forums and used for CNP fraud - enumeration attacks testing card validity, followed by high-value purchases at electronics merchants.

- Batch Scoring Gap Exploitation: Fraudsters had reverse-engineered the 15-minute batch scoring window and timed their transactions to occur just after a batch run, giving them a nearly 15-minute window before their transactions would be scored. During this window, they could execute 10-20 fraudulent transactions before any alert was raised.

The financial impact was severe: fraud losses increased from 12 crore INR per quarter to 53 crore INR in Q3 2025. The false positive rate spiked from 0.3% to 2.1%, causing customer complaints to triple. The board mandated a complete platform re-architecture, with the Platform Architect (the intern) tasked to deliver the architectural design in 15 days.

B1.3 The Risk & Technology Board: Evaluation Framework

The intern's architectural design will be evaluated by a simulated Risk & Technology Board comprising five members, each applying a different lens:

- Chief Technology Officer (CTO): Evaluates architectural soundness (are service boundaries clean?), technology choices (are the chosen databases, message brokers, and frameworks appropriate?), scalability (can the system handle 10x current load?), and technical debt management (does the migration strategy avoid accumulating new debt?).
- Chief Risk Officer (CRO): Evaluates fraud detection capability (are all known attack vectors covered?), regulatory compliance (does the system meet PCI DSS, RBI, and GDPR requirements?), risk coverage (is the system resilient to new, unknown attack types through ML and graph analysis?), and decision explainability (can every fraud decision be explained to a regulator?).
- VP of Engineering: Evaluates operational feasibility (can a team of 30–40 engineers build and maintain this?), deployment strategy (are deployments safe and reversible?), team structure (can the microservices be assigned to teams of 3–5 developers each?), and developer experience (are the tooling and workflows reasonable?).
- Head of Compliance: Evaluates audit trail completeness (is every action logged immutably?), data privacy (is cardholder data encrypted and access-controlled?), regulatory reporting (can the system generate RBI fraud reports, PCI DSS audit evidence, and GDPR data subject access requests?), and data retention (does the system comply with retention requirements across jurisdictions?).
- Chief Financial Officer (CFO): Evaluates cost efficiency (what is the estimated monthly infrastructure cost?), ROI projection (what is the expected reduction in fraud losses?), build vs buy decisions (should any component be sourced from a vendor?), and scalability economics (does cost scale linearly or sub-linearly with transaction volume?).

B1.4 Progressive Challenge Levels

The simulation is structured into six escalating challenge levels, each introducing additional complexity. Points are awarded at each level based on the quality of deliverables:

Level	Focus	Days	Points	Key Deliverables
1	Decomposition	1-3	150	Bounded Context Map, C4 Level 1/2/3, Service table, SLAs
2	Communication	4-6	200	Kafka topology, Event schemas, API contracts (OpenAPI + Protobuf), EDA spec, Sagas
3	Detection Engineering	7-9	200	Rule engine framework, ML serving architecture, Graph schema & queries
4	Resilience & Security	10-11	150	Istio config, mTLS, encryption strategy, API gateway, rate limiting
5	Observability & Ops	12-13	150	Grafana dashboards, Prometheus alerts, logging spec, tracing, SLA doc
6	Deployment & Present	14-15	150	CI/CD pipeline, DR plan, failover runbook, Board presentation

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B2: ACHIEVEMENT BADGES AND BONUS POINTS

Interns who demonstrate exceptional work in specific areas earn achievement badges that are noted in their assessment report. Each badge carries recognition but no additional points beyond what is awarded in the rubric. Badges are:

- Decomposition Master - All 8-12 services have clearly justified boundaries with no circular dependencies and explicit trade-off analysis for each boundary decision.
- Event Storm Champion - Event Storming diagram covers 50+ domain events with complete aggregate mapping, policy definitions, and external system integration points.
- Latency Hunter - Latency budget is specified per service per hop with mathematical justification (measured or estimated latency for each component: network hop, serialisation, processing, database query).
- Graph Sleuth - Graph analysis design includes three or more novel query patterns for fraud ring detection with worked examples showing sample data and expected results.
- Compliance Architect - Architecture addresses PCI DSS, RBI guidelines, and GDPR simultaneously with explicit mapping of each requirement to the architectural component that satisfies it.
- Chaos Engineer - Design includes failure injection strategy with at least 5 chaos experiments, expected behaviour under each failure, and the monitoring signals that would detect it.
- Zero-Downtime Deployer - Deployment strategy demonstrates provably zero message loss during rolling updates with mathematical proof or simulation evidence.
- Cost Optimizer - Architecture includes a detailed cost estimation for cloud deployment (compute, storage, network, managed services) with a cost-per-transaction metric and comparison against the legacy system cost.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART C: CASE STUDIES & REAL-WORLD FRAUD INCIDENTS

This section presents four detailed case studies drawn from real-world fraud incidents, architectural failures, and successful detection systems. Each case study includes a simulation scenario that the intern must analyse and incorporate into their architectural design. The case studies are selected to represent different failure modes: detection latency, real-time response failure, resilience at scale, and audit/compliance failure.

SECTION C1: THE TARGET DATA BREACH (2013) - LESSONS IN DETECTION LATENCY

In November–December 2013, attackers infiltrated Target Corporation's network through a third-party HVAC vendor (Fazio Mechanical Services) that had network access for billing and contract purposes. The attackers installed memory-scraping malware (a variant of BlackPOS) on point-of-sale terminals across 1,797 stores, exfiltrating 40 million credit and debit card numbers and 70 million customer records (names, addresses, phone numbers, email addresses) over a 19-day period before the breach was detected and contained.

Target's security monitoring system (FireEye) actually generated alerts during the breach. Bangalore-based security analysts flagged the alerts and escalated to Target's Minneapolis headquarters. However, the alerts were not acted upon due to: alert fatigue (the security operations centre received thousands of alerts daily, many low-priority), poor integration between the detection system and the response workflow (FireEye alerts required manual triage and did not auto-create investigation tickets), absence of automated escalation for high-severity alert patterns (the system could not distinguish between routine malware detections and a coordinated exfiltration campaign), and lack of automated response capabilities (even after detection, stopping the breach required manual intervention on 1,797 POS terminals).

Architectural Lessons For SWE-2C: This case demonstrates the critical importance of: (1) Automated escalation paths - the system must act on high-confidence alert patterns without waiting for human review. When multiple POS terminals in the same store generate malware alerts within an hour, the system should auto-escalate to P1 and auto-create an investigation case. (2) Alert correlation and deduplication - the alerting system must correlate related alerts (same store, same time window, same malware signature) into a single incident rather than generating independent alerts for each terminal. (3) Integration between detection and response - the Case Management service must be tightly coupled with the alerting pipeline to auto-create and prioritise investigation cases with all relevant context pre-populated. (4) Third-party risk management - the API gateway must enforce strict access controls and monitoring for third-party integrations, including anomaly detection on API usage patterns (a vendor's API client suddenly accessing data outside normal business hours should trigger an alert). (5) Automated containment - the system should have the capability to auto-block compromised terminals or freeze specific card ranges pending investigation.

Simulation Scenario: Design the alerting and escalation pipeline that would have detected and escalated the Target-style breach within minutes rather than days. Specifically: (a) Design the alert correlation engine that groups POS terminal malware alerts by store_id, time_window, and malware_signature_hash, and creates a composite incident when the correlated count exceeds a threshold. (b) Design the automated escalation rule that pages the security team when the

composite incident affects more than 3 stores or more than 50 terminals. (c) Design the circuit-breaker mechanism that automatically disables compromised POS terminal integration points while permitting legitimate transactions through alternative channels.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C2: THE COSMOS BANK CYBER ATTACK, INDIA (2018) - REAL-TIME RESPONSE FAILURE

In August 2018, Cosmos Cooperative Bank in Pune, India, suffered a sophisticated malware attack on its ATM server infrastructure. The attackers compromised the switch that connects the bank's core banking system to the ATM network and the SWIFT messaging system. They created a proxy switch that intercepted and approved ATM withdrawal requests without routing them to the core banking system for balance verification. Over a span of two days (11-13 August), approximately 94.42 crore INR (approximately 13.5 million USD) was siphoned through 14,849 fraudulent ATM withdrawals across 28 countries using cloned cards, and through three unauthorised SWIFT transfers totalling 13.92 crore INR to accounts in Hong Kong.

Architectural Lessons for SWE-2C: (1) End-to-end integrity verification: Every transaction must be verified against the authoritative source of truth (core banking system). A proxy or bypass must be detectable through integrity checksums - the system should verify that every ATM authorisation request was actually processed by the core banking system by comparing request logs between the switch and the CBS. (2) Geographic velocity checking: 14,849 ATM withdrawals across 28 countries in approximately 36 hours is a pattern that even a simple velocity rule would flag. The system must maintain per-card windowed counters partitioned by country and trigger alerts for impossible geographic patterns. (3) SWIFT message anomaly detection: Three large international SWIFT transfers from a cooperative bank to Hong Kong within hours represent a massive deviation from normal SWIFT messaging patterns. A statistical anomaly detector monitoring SWIFT message volume, destination countries, and amounts would flag this immediately. (4) Defence-in-depth architecture: The fraud detection system must not rely on a single point of control (the switch) that can be bypassed by malware. Independent monitoring layers must validate transactions at multiple points in the flow.

Simulation Scenario: Design: (a) The geographic velocity service that detects simultaneous ATM withdrawals from the same card across multiple countries. Specify the Kafka consumer that maintains a time-windowed count of ATM transactions per card, partitioned by country, and triggers a P1 alert when transactions from more than 2 countries occur within a 60-minute window. (b) The SWIFT message monitoring service that compares each SWIFT transfer against the bank's 12-month historical transfer profile (destination countries, amounts, frequency) and flags deviations exceeding 3 standard deviations. (c) The integrity verification layer that independently validates every transaction authorisation by cross-referencing switch logs with core banking system logs, flagging discrepancies within 30 seconds.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C3: NETFLIX'S MICROSERVICES ARCHITECTURE - RESILIENCE AT SCALE

While not a fraud detection case, Netflix's microservices architecture is the gold standard for designing resilient distributed systems at scale and provides directly applicable patterns. Netflix serves over 260 million subscribers across 190+ countries with more than 700 microservices, processing millions of API requests per second. Key architectural patterns that interns must evaluate for the fraud detection platform include:

- Hystrix / Resilience4j (Circuit Breaker Pattern): Prevents cascading failures when a downstream service is unhealthy. When the error rate for calls to a service exceeds a threshold (e.g., 50% over a 10-second window), the circuit opens and subsequent calls are immediately failed or routed to a fallback, giving the downstream service time to recover. The circuit automatically closes after a configurable recovery period.

- Zuul / Spring Cloud Gateway (Edge Proxy): Handles request routing, load balancing, authentication, rate limiting, and static response handling at the edge. This maps directly to the API gateway in the fraud detection platform.

- Eureka (Service Discovery): Enables dynamic service registration and DNS-based discovery. Services register themselves on startup and deregister on shutdown. Client-side load balancing uses the service registry to find available instances. In a Kubernetes environment, this is partially replaced by Kubernetes Services and DNS, but the principle of dynamic discovery remains.

- Chaos Monkey and the Simian Army: Proactively inject failures into the production environment to test resilience. Chaos Monkey randomly terminates instances; Latency Monkey injects artificial delays; Chaos Gorilla simulates entire availability zone failures. This philosophy of "designing for failure" is essential for a fraud detection platform where downtime means either lost transactions (revenue impact) or unscreened transactions (fraud impact).

Architectural Lessons: Netflix's approach of designing for failure is directly applicable. The fraud detection system must continue to make fraud decisions even when individual services are degraded. A fallback strategy must be defined for each service: if the ML model service is unavailable, the system falls back to rule-only scoring with a lower approval threshold (more conservative); if the graph service is unavailable, the system proceeds without graph signals but adjusts the risk score upward by a configurable safety margin; if the feature store (Redis) is unavailable, the system uses default feature values (e.g., population averages) instead of personalised features. The intern must document fallback strategies for every service, specifying the fallback behaviour, the impact on detection accuracy, and the monitoring alert that will fire when a fallback is activated.

SECTION C4: THE WIRECARD SCANDAL (2020) - AUDIT AND COMPLIANCE FAILURES

Wirecard, a German payment processing and financial services company valued at 24 billion EUR at its peak and a member of the DAX 30 index, collapsed in June 2020 when auditor EY could not confirm the existence of 1.9 billion EUR in cash balances purportedly held at two Philippine banks (BPI and BDO Unibank). The money did not exist. The fraud had persisted for years through fabricated audit documentation (forged bank statements and trustee confirmations), inadequate segregation of duties in financial reporting (the same team that booked revenue also controlled the "trust accounts"), and regulatory blind spots (BaFin, the German financial regulator, initially investigated Financial Times journalists who were reporting on the fraud rather than investigating the company, even banning short-selling of Wirecard stock).

Architectural Lessons for SWE-2C: This case underscores the absolute necessity of immutable audit trails in any financial system. The Audit service in the fraud detection platform must: (1) Write to an append-only data store (e.g., Amazon QLDB, Apache Kafka with infinite retention and immutable topics, or a custom implementation using a Merkle tree hash chain) that cannot be modified or deleted by any application, administrator, or even database superuser. (2) Log every fraud decision (with the complete input data, all rule results, ML scores, graph signals, and final decision), every rule change (old version, new version, author, approval chain), every model deployment (old model, new model, validation metrics, approver), every threshold modification, every user access to cardholder data, and every configuration change. (3) Cryptographically hash each audit entry and chain it to the previous entry (Merkle tree) so that any tampering is detectable by verifying the hash chain from the genesis block. (4) Provide regulatory query endpoints that allow an RBI auditor or PCI DSS assessor to independently retrieve and verify audit records for a specified time period, including hash chain validation. (5) Implement retention policies that comply with RBI (minimum 7 years for transaction records), PCI DSS (minimum 1 year for detailed audit trails), and GDPR (right to erasure for personal data - handled by retaining the audit trail but pseudonymising personal identifiers after the retention period expires).

Simulation Scenario: Design the Audit and Compliance service including: (a) The audit event schema with all mandatory fields (event_type, timestamp, actor_id, actor_role, resource_type, resource_id, action, before_state, after_state, justification, trace_id, hash, previous_hash). (b) The storage architecture (append-only with cryptographic integrity verification and hash chain). (c) The query patterns for regulatory examination (retrieve all decisions for a card in a date range, retrieve all rule changes by a specific author, retrieve all model deployments with their validation metrics). (d) The retention and pseudonymisation strategy for GDPR compliance. (e) The access control policies ensuring that no single administrator can modify audit records (requiring dual authorisation for any maintenance operation on the audit store).

PART D: 15-DAY SPRINT METHODOLOGY & DAILY DELIVERABLES

This part specifies the day-by-day methodology for the 15-day sprint. Each day comprises 8 working hours (total: 120 hours). The methodology follows a progression from domain understanding through architectural design to implementation specifications and final presentation. All deliverables must be committed to the GitHub repository daily by 11:59 PM IST. Late commits incur a 2% penalty per day on that day's deliverables.

SECTION D1: PHASE 1 - DOMAIN UNDERSTANDING AND DECOMPOSITION (DAYS 1–3)

Day 1 - Domain Immersion and Monolith Analysis:

Learning Goals: Understand fraud detection domain concepts thoroughly (CP vs CNP fraud types, rule-based detection, anomaly detection, graph analysis, risk scoring, regulatory requirements). Analyse the legacy monolith architecture provided in the simulation scenario. Identify business capabilities and initial bounded context candidates.

- Study Part A Sections A1 and A2 thoroughly, taking notes on key concepts.
- Create a domain glossary of 50+ terms specific to fraud detection (e.g., PAN, BIN, MCC, AFA, tokenisation, chargeback, false positive rate, detection rate, SIM-swap, enumeration attack, synthetic identity, money mule).
- Review the legacy monolith system description in Section B1.1.
- Map each functional area of the monolith to a business capability (e.g., "rule evaluation logic in Java switch statements" maps to the "Fraud Rule Evaluation" business capability).
- Document all external system integrations (Visa/Mastercard/RuPay payment networks, core banking system, customer mobile app, regulatory reporting endpoints).
- Conduct an Event Storming session for the monolith's current functionality using Miro or Excalidraw.
- Start with the transaction lifecycle: a transaction is received from a payment channel, enriched with customer and device data, evaluated by rules, scored by the batch ML model, flagged or approved, notified to the customer (if flagged), assigned to an analyst (if for review), investigated, and resolved.
- Identify all domain events (transaction_received, transaction_enriched, rule_evaluated, ml_scored, risk_decision_made, case_created, case_assigned, case_resolved, notification_sent, etc.).
- Map events to aggregates (Transaction, FraudCase, CustomerProfile, Rule, etc.) and commands (SubmitTransaction, EvaluateRules, CreateCase, AssignAnalyst, etc.).

- Identify pain points: single database bottleneck, batch ML scoring, hard-coded rules, no graph analysis, manual deployments.

Deliverables: (1) Domain glossary document (Markdown, minimum 50 terms with definitions). (2) Legacy monolith analysis document identifying all business capabilities, external integrations, and architectural pain points. (3) Initial Event Storming output (Miro/Excalidraw export or Draw.io diagram showing events, commands, aggregates). (4) Work log entry for Day 1. Git commit: Day01_Domain_Immersion.

Day 2 - Bounded Context Identification and Service Decomposition:

Learning Goals: Apply DDD principles to define bounded contexts. Decide service granularity (target: 8-12 services). Identify context mapping patterns between services. Produce the C4 Level 1 System Context diagram.

- Using the Event Storming output from Day 1, group related events and aggregates into bounded contexts.
- Apply the following heuristics: business capability alignment (each service maps to exactly one business capability), data ownership (each service owns its data and exposes it only through APIs), team alignment (each service can be owned by a team of 3-5 developers), change frequency (parts that change together should be in the same service), scalability requirements (parts with different scaling profiles should be separate services), and failure domain isolation (a failure in graph analysis should not bring down transaction ingestion).
- Document each bounded context with its name, responsibilities, owned data, exposed API, and consumed events.
- Produce the C4 Level 1 (System Context) diagram showing the fraud detection platform and all external systems with their communication protocols and data flow directions.
- Begin the C4 Level 2 (Container) diagram listing all microservices, databases, message brokers, API gateway, and monitoring components.
- For each service, document: service name, bounded context, primary responsibility, technology choice (language, framework), data store (database type, schema summary), and team ownership (which team would own this service in a real organisation).
- Create the service decomposition table.

Deliverables: (1) Bounded Context Map document with all contexts and their relationships. (2) C4 Level 1 System Context diagram (Draw.io or Mermaid, exported as SVG/PNG). (3) Draft C4 Level 2 Container diagram. (4) Service decomposition table. Git commit: Day02_Service_Decomposition.

Day 3 - Container and Component Diagrams, SLAS, and Persistence:

Learning Goals: Complete C4 Level 2 and Level 3 diagrams. Define data ownership and polyglot persistence strategy. Specify service-level SLAs.

- Finalise the C4 Level 2 Container diagram with all inter-service communication flows: synchronous gRPC calls shown as solid arrows with the RPC method name, asynchronous Kafka events shown as dashed arrows with the topic name.
- For each of the four core services (Transaction Ingestion, Rule Engine, Anomaly Detection, Risk Scoring), create a C4 Level 3 Component diagram showing internal architecture: API handlers (REST controllers or gRPC server), domain service layer (business logic), repository/data access layer, integration adapters (Kafka producers/consumers, gRPC clients), and infrastructure components (health check endpoint, metrics exporter, configuration loader).
- Define the SLA for each service as a table: service name, maximum latency (p50, p95, p99), availability target (percentage), throughput target (TPS), error budget policy (what happens when the error budget is exhausted - e.g., freeze deployments until error budget recovers), and dependency SLAs (what SLAs this service depends on from upstream services).
- Document the polyglot persistence strategy: PostgreSQL for Transaction Ingestion (ACID transactions, strong consistency for financial data), Redis for Feature Store and rate limit counters (sub-millisecond reads), Neo4j for Graph Analysis (native graph storage for relationship queries), TimescaleDB or Cassandra for time-series analytics (high write throughput, time-based partitioning), and Elasticsearch for log storage and full-text search (inverted index for fast log queries).

Deliverables: (1) Finalised C4 Level 2 Container diagram. (2) C4 Level 3 Component diagrams for 4 core services. (3) Service SLA specification table. (4) Polyglot persistence justification document. Git commit: Day03_C4_Diagrams.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION D2: PHASE 2 - COMMUNICATION AND DETECTION DESIGN (DAYS 4-9)

Day 4 - Kafka Topic Design and Event Schema:

Learning Goals: Design the complete Kafka topic topology. Define event schemas using Avro or Protobuf. Specify partitioning strategies and consumer groups.

- Design the Kafka topic topology.
- For each topic, specify: topic name (following naming convention: domain.entity.action, e.g., fraud.transactions.raw), partition count (calculated based on target throughput and consumer parallelism), replication factor (3 for production, 1 for development), retention policy (fraud.audit.events: infinite retention; fraud.transactions.raw: 30 days; fraud.notifications: 7 days), key schema (e.g., card_number_hash for transaction topics to ensure per-card ordering), and consumer groups (which services consume from this topic and their consumer group IDs).
- Design at least 8 topics covering the complete transaction lifecycle.
- Define Avro or Protobuf schemas for all event types: TransactionEvent (raw transaction from payment channel), EnrichedTransactionEvent (enriched with device, geo, and customer profile), RuleEvaluationResult (per-rule trigger/pass result), AnomalyScore (ML model prediction with feature contributions), GraphSignals (relationship analysis results), RiskDecision (composite score and action), CaseEvent (case lifecycle events), NotificationRequest (multi-channel notification), and AuditEvent (immutable audit log entry).
- Each schema must include: schema versioning strategy (backward-compatible evolution using optional fields and defaults), Schema Registry configuration (Confluent Schema Registry or equivalent), compatibility level setting (BACKWARD for most topics, FULL for audit events), and dead-letter queue (DLQ) topology for handling poison messages (messages that fail deserialization or processing after N retries).

Deliverables: (1) Kafka topic topology document. (2) Event schema definitions (Avro .avsc or Protobuf .proto files). (3) Schema Registry configuration. (4) DLQ strategy document. Git commit: Day04_Kafka_Design.

Day 5 - API Contract Design (OPENAI and Protobuf)

- Create OpenAPI 3.0 specifications for all external-facing REST APIs exposed through the API gateway.
- Each endpoint must specify: path and HTTP method, request body schema with all fields typed and described, response schemas for success (2xx) and error (4xx, 5xx) cases, authentication requirements (JWT, API key, or OAuth 2.0), rate limit policies (requests per minute), and example request/response pairs.

- Minimum endpoints: POST /api/v1/transactions, GET /api/v1/transactions/{id}/risk-score, GET /api/v1/cases (paginated, filterable), PUT /api/v1/cases/{id}/decision, GET /api/v1/rules (paginated), POST /api/v1/rules (create new rule), GET /api/v1/dashboard/metrics (aggregated statistics).
- Create Protobuf service definitions for internal gRPC APIs.
- Define services: RuleEngineService (Evaluate RPC), AnomalyDetectionService (Score RPC), GraphAnalysisService (AnalyseRelationships RPC), RiskScoringService (ComputeScore RPC), CustomerProfileService (GetProfile, UpdateProfile RPCs), and FeatureStoreService (GetFeatures RPC).
- Each RPC must specify: request and response message types with all fields, deadline/timeout (e.g., 50ms for RuleEngine.Evaluate), retry policy (max retries, backoff strategy), and error codes (following gRPC status codes).
- Design the API gateway routing table mapping external REST paths to internal gRPC services with request transformation rules.

Deliverables: (1) OpenAPI 3.0 specification file (YAML). (2) Protobuf .proto files for all internal gRPC services. (3) API gateway routing configuration. (4) Authentication and authorisation scheme documentation. Git commit: Day05_API_Contracts.

Day 6 - Event-Driven Architecture Specification:

- Create the comprehensive Event Storming diagram covering all domain events (minimum 50), commands, aggregates, policies, read models, and external systems.
- This diagram is the single source of truth for the event-driven architecture.
- Identify and document all saga workflows: Transaction Processing Saga (ingestion → enrichment → rule evaluation → ML scoring → graph analysis → risk scoring → decision → notification), Fraud Investigation Saga (alert → case creation → card freeze → analyst assignment → investigation → decision → card unfreeze/permanent block → chargeback initiation), and Card Blocking Saga (fraud confirmed → block card → issue replacement → notify customer → update profile).
- Design CQRS read models for the Analytics Dashboard: (1) Real-time fraud statistics materialised view (transactions processed, approved, declined, flagged for review, by hour, by channel, by market), (2) Geographic heat map materialised view (fraud attempts by geographic region, updated every minute), (3) Rule effectiveness materialised view (per rule: trigger count, true positive count, false positive count, detection rate, false positive rate), (4) Model performance materialised view (per model version: AUC, precision, recall, F1 score, computed daily), (5) Case management backlog materialised view (open cases by priority, by analyst, average resolution time).

- Specify the Kafka Streams or ksqlDB topology for each materialised view including the source topic(s), processing logic, and sink (database table or topic).

Deliverables: (1) Complete Event Storming diagram with 50+ events. (2) Saga orchestration diagrams for all sagas with compensation logic. (3) CQRS read model specifications with materialisation topology. Git commit: Day06_EDA_Specification.

Day 7 - Rule Engine Configuration Framework:

- Design the rule configuration schema (YAML/JSON) supporting: conditions (field path, operator [eq, neq, gt, gte, lt, lte, in, not_in, between, regex], value, temporal_window [count, sum, avg, max, min over a duration]), logical operators (AND, OR, NOT with unlimited nesting), entity scopes (cardholder, merchant, terminal, device, network), actions (flag, block, score_adjust with delta, notify with channel and template, step_up_auth with method), metadata (rule_id, version, author, created_at, effective_date, expiry_date, severity [low/medium/high/critical], description, category), and performance constraints (max_evaluation_time_ms).
- Provide a JSON Schema for validation. Create 20 sample rules covering: velocity checks (5 rules), amount thresholds (3 rules), geographic anomalies (3 rules), MCC restrictions (3 rules), watchlist matching (2 rules), behavioural anomalies (2 rules), and device/network risks (2 rules).
- Design the rule lifecycle management system: (1) Rule authoring workflow (draft → review → approved → active → deprecated → retired) with role-based permissions (analyst creates, manager reviews, director approves). (2) Rule versioning (each change creates a new version; old versions are archived, never deleted; rollback to any previous version is supported). (3) Rule simulation (backtest a rule against 30 days of historical data, producing a report: transactions evaluated, transactions flagged, confirmed fraud in flagged set, confirmed legitimate in flagged set, detection rate, false positive rate, estimated annual fraud prevented, estimated annual customer friction). (4) Rule A/B testing (run a new rule in shadow mode alongside existing rules, compare outcomes over 7 days before full activation). (5) Rule performance dashboard (real-time per-rule metrics: trigger rate, true positive rate, false positive rate, evaluation latency).

Deliverables: (1) Rule engine configuration schema (YAML specification with JSON Schema validation). (2) 20 sample rule definitions. (3) Rule lifecycle state machine diagram. (4) Rule simulation specification. (5) Rule performance monitoring design. Git commit: Day07_Rule_Engine_Design.

Day 8 - ML Model Serving and Anomaly Detection Architecture:

- Design the ML model serving architecture including: (1) Model registry (MLflow or custom) for version management with metadata (model type, training date, training dataset size, validation metrics, approver, deployment date). (2) Model deployment pipeline (train on historical data → validate on holdout set → stage in pre-production → canary deployment at 5% traffic → full production). (3) Inference engine selection with justification (ONNX Runtime for cross-framework portability, TensorFlow Serving for TF models, or custom FastAPI server). (4) Model ensemble strategy (how to combine scores from the supervised model, unsupervised model, and sequence model). (5) Feature Store design with two tiers: batch features (computed daily by a Spark job, stored in PostgreSQL, synced to Redis at midnight) and real-time features (computed per-transaction by a Kafka Streams application, written to Redis with TTL).
- Design the model monitoring pipeline: (1) Data drift detection using Population Stability Index (PSI) on input feature distributions, computed hourly, with alert threshold at PSI > 0.25. (2) Prediction drift detection monitoring the distribution of risk scores over time using the Kolmogorov-Smirnov test. (3) Performance monitoring comparing model predictions against confirmed fraud labels (available with a 30–90 day lag from chargeback data) with automated performance reports. (4) Automated retraining triggers (retrain when PSI > 0.25, or when weekly detection rate drops below 90%, or on a fixed 90-day cadence). (5) Champion-challenger framework: the current production model is the "champion"; new candidate models are "challengers" that run in shadow mode, receiving the same input but not influencing the decision, with their predictions compared against the champion's over a minimum 7-day evaluation period.

Deliverables: (1) ML model serving architecture document. (2) Feature store design with batch and real-time feature definitions and data flow diagrams. (3) Model monitoring specification with drift detection methodology. (4) Champion-challenger framework document. Git commit: Day08_ML_Architecture.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Day 9 - Graph-Based Fraud Detection Design:

- Design the property graph schema: node types with their properties (Card: card_hash, bin, issuer, product_type, created_at; Account: account_id, type, status, opened_at; Customer: customer_hash, risk_tier; Device: device_fingerprint, os, browser, first_seen; IPAddress: ip_hash, country, city, is_vpn, is_proxy, is_tor; PhoneNumber: phone_hash, carrier, country; Email: email_hash, domain, disposable; PhysicalAddress: address_hash, city, state, country, zip; Merchant: merchant_id, mcc, name, risk_category), and edge types with their properties (TRANSACTED_WITH: timestamp, amount, channel; SHARED_DEVICE: first_shared_at, last_shared_at; SHARED_ADDRESS: relationship_type; SAME_BENEFICIARY: transfer_count; LINKED_ACCOUNT:

link_type; USED_IP: timestamp, session_duration; REGISTERED_PHONE: registration_date; REGISTERED_EMAIL: registration_date).

- Design at least 5 Cypher query patterns with sample queries, expected input, and expected output for each.
- Design the graph synchronisation strategy: (1) Kafka consumer reading from transactions.enriched topic and updating the graph with new nodes and edges (specify the graph update logic: create node if not exists, create or update edge with latest transaction data). (2) Batch reconciliation job (daily, compares graph state against the transactional database, identifies and repairs inconsistencies). (3) Graph pruning strategy (remove edges older than 180 days to prevent graph bloat; archive pruned data to cold storage). (4) Read-replica strategy (separate read replica for analytical queries like community detection that run for minutes, preventing them from blocking real-time lookups that must complete in milliseconds). (5) Graph index design (indexes on card_hash, device_fingerprint, ip_hash for fast lookups; full-text index on merchant_name for fuzzy matching).

Deliverables: (1) Graph database schema (all nodes, edges, properties, indexes). (2) 5+ Cypher query patterns with sample queries and expected results. (3) Graph synchronisation architecture. (4) Graph pruning and maintenance strategy. Git commit: Day09_Graph_Analysis_Design.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION D3: PHASE 3 - RESILIENCE, OBSERVABILITY, AND DEPLOYMENT (DAYS 10-15)

Day 10 - Service Mesh and Security Design:

- Design complete Istio configuration (DestinationRules, VirtualServices, AuthorizationPolicies, PeerAuthentication).
- Create the service communication matrix.
- Design data encryption strategy for PCI DSS (AES-256 at rest, mTLS in transit, PAN tokenisation, key rotation via Vault).
- Map each PCI DSS requirement to the architectural component that satisfies it.

Deliverables: Istio configuration files, service communication matrix, encryption strategy document, PCI DSS compliance mapping. Git commit: Day10_Security_Design.

Day 11 - API Gateway and Rate Limiting Design:

- Design API gateway configuration (routing, authentication, request transformation).
- Design tiered rate limiting framework (per-merchant, per-IP, per-endpoint, global, adaptive).
- Design circuit breaker configuration at gateway level.

Deliverables: API gateway configuration document, rate limiting policy specification, authentication flow diagrams. Git commit: Day11_Gateway_Design.

Day 12 - Monitoring Dashboard and Alerting Specification:

- Design four Grafana dashboards (Transaction Processing, Fraud Detection, Case Management, Infrastructure).
- Specify Prometheus recording rules and alerting rules with severity-based routing.
- Create runbooks for P1 and P2 alerts.

Deliverables: Grafana dashboard specifications, Prometheus alerting rules, runbook templates. Git commit: Day12_Monitoring_Design.

Day 13 - Logging, Tracing and SLA Specification:

- Design structured logging standard (JSON format, mandatory fields, PII masking rules, retention policy).
- Configure OpenTelemetry tracing (sampling strategy, custom span attributes, storage).
- Formalise SLA document (all targets, measurement methodology, error budget policy).

Deliverables: Structured logging specification, tracing configuration, SLA specification document. Git commit: Day13_Observability_Design.

Day 14 - CI/CD Pipeline and Deployment Strategy:

- Design CI/CD pipeline per service (source, build, test, scan, stage, canary, production stages).
- Design multi-region deployment topology with disaster recovery.
- Create failover runbook with RPO/RTO specifications.
- Write sample Dockerfiles and Kubernetes manifests for 2 services.

Deliverables: CI/CD pipeline design, deployment topology diagram, DR plan, failover runbook, sample Dockerfiles/K8s manifests. Git commit: Day14_CICD_Deployment.

Day 15 - Final Assembly, Review, and Board Presentation:

- Assemble the Master Architecture Document consolidating all deliverables with table of contents, cross-references, and consistent formatting.
- Self-review against the 1000-point rubric.
- Ensure diagram consistency (service names, data flows match across all documents).
- Check for the 5 embedded errors in this project document and document findings in /docs/ERROR_DETECTION.md.
- Prepare a 15-minute Board Presentation covering: Executive Summary, Architecture Overview (C4 diagrams), Detection Strategy (rules + ML + graph), Resilience and Security, Operational Readiness, and Risk Mitigation.
- Record a video walkthrough (optional, bonus).
- Ensure README.md provides clear repository navigation.

- Transfer repository ownership to ZethetaIntern.

Deliverables: Master Architecture Document, Board Presentation, README.md, final Git commit: Day15_Final_Assembly. Repository transfer to ZethetaIntern.

Submission Instructions:

- All files to be named using convention: ProjectCode_YourName_DeliverableName (e.g., 201169C_RahulShah_MainReport)
- All PDFs must be readable and not password-protected
- Supporting Excel files must be in .xlsx format
- All daily deliverables across Days 1–15 are to be submitted on **Day 15** as a consolidated final submission. Students should combine their work into **one or two PDFs and/or one Excel file** wherever possible, rather than submitting separate files for each day. Clearly label each section within the consolidated document by its respective day and topic for ease of assessment.
- Late submissions attract the penalties specified in Part F Assessment Criteria

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION D4: DAILY DELIVERABLE SUMMARY TABLE

Day	Focus Area	Primary Deliverables	Commit Tag
1	Domain Immersion	Glossary (50+ terms), Monolith Analysis, Event Storming	Day01_Domain_Immersion
2	Service Decomposition	Bounded Context Map, C4 Level 1, Service Table	Day02_Service_Decomposition
3	C4 Diagrams & SLAs	C4 Level 2 & 3, SLA Table, Persistence Strategy	Day03_C4_Diagrams
4	Kafka Design	Topic Topology (8+ topics), Event Schemas, DLQ Strategy	Day04_Kafka_Design
5	API Contracts	OpenAPI 3.0 Spec, Protobuf .proto files, Gateway Config	Day05_API_Contracts
6	Event-Driven Arch	Event Storming (50+ events), Sagas, CQRS Read Models	Day06_EDA_Specification
7	Rule Engine Design	Rule Schema + 20 Rules, Lifecycle Management, Simulation	Day07_Rule_Engine_Design
8	ML Architecture	Model Serving, Feature Store, Monitoring, Champion-Challenger	Day08_ML_Architecture
9	Graph Analysis	Graph Schema, 5+ Cypher Queries, Sync Strategy	Day09_Graph_Analysis_Design
10	Security Design	Istio Config, Encryption, PCI DSS Mapping	Day10_Security_Design
11	API Gateway	Gateway Config, Rate Limiting (5 tiers), Auth Flows	Day11_Gateway_Design
12	Monitoring	4 Grafana Dashboards, Alert Rules, Runbooks	Day12_Monitoring_Design
13	Observability & SLAs	Logging Spec, Tracing Config, SLA Document	Day13_Observability_Design
14	CI/CD & Deployment	Pipeline Design, Multi-Region Topology, DR Plan	Day14_CICD_Deployment
15	Final Assembly	Master Document, Board Presentation, README, Transfer	Day15_Final_Assembly

PART E: DEVELOPMENT TOOLS, FRAMEWORKS & PLATFORM GUIDE

This section provides a comprehensive guide to the tools, frameworks, and platforms that interns should use for this project. This is a Software Engineer project requiring professional engineering tools. All tools listed are free or have free tiers sufficient for this project.

SECTION E1: DIAGRAMMING AND DESIGN TOOLS

Tool	Purpose	URL	Usage Notes
Draw.io / diagrams.net	C4 model diagrams, deployment topology	https://app.diagrams.net	Import C4 shape library from Structurizr. Export as SVG and PNG.
Mermaid	Sequence diagrams, flowcharts in Markdown	https://mermaid.js.org	Renders natively in GitHub Markdown. Ideal for in-repo documentation.
Excalidraw	Whiteboard-style sketches	https://excalidraw.com	Event Storming sessions, early brainstorming.
Miro	Event Storming templates	https://miro.com	Free tier. Sticky note format for event identification.
PlantUML	UML diagrams as code	https://plantuml.com	Component and class diagrams. Integrates with VS Code.

SECTION E2: API DESIGN AND DOCUMENTATION

Tool	Purpose	URL
Swagger Editor	OpenAPI 3.0 spec authoring with validation	https://editor.swagger.io
Buf	Protobuf linting and breaking change detection	https://buf.build
Postman / Insomnia	API testing against mock servers	https://postman.com
Redoc	API documentation rendering from OpenAPI	https://redocly.com/redoc

SECTION E3: VERSION CONTROL AND REPOSITORY STRUCTURE

REPOSITORY NAME: SWE-2C_FraudDetection_[YourFullName]

REQUIRED DIRECTORY STRUCTURE:

- /docs - All design documents, architecture specifications, and the Master Architecture Document.
- /diagrams - All C4 diagrams, Event Storming exports, deployment topology, communication flows (source + exported PNG/SVG).
- /api-specs - OpenAPI YAML files and Protobuf .proto files.
- /configs - Rule engine YAML, Kafka topic config, Istio policies, Prometheus alert rules, Grafana dashboard JSON.
- /samples - Sample Dockerfiles, Kubernetes manifests, Helm charts (for 2+ services).
- /presentation - Board presentation slide deck and optional video walkthrough.
- /daily-commits - Daily summary logs: work done, decisions made, blockers.
- README.md - Repository navigation guide with links to all key documents.

COMMIT DISCIPLINE: Minimum one meaningful commit per day using Conventional Commits format (docs:, feat:, fix:, refactor:). Empty or trivial commits (“update”, “fix”) incur point deductions.

SECTION E4: INFRASTRUCTURE AND PLATFORM TOOLS

Tool	Usage in This Project	Required?
Docker	Sample Dockerfiles for 2+ services (multi-stage, distroless)	Yes (design + samples)
Kubernetes	Sample manifests (Deployment, Service, HPA, PDB, NetworkPolicy)	Yes (design + samples)
Helm	Parameterised deployment for 1 service	Optional (bonus)
Apache Kafka	Topic topology design; optional local Docker Compose for validation	Yes (design); optional (local)
Neo4j Sandbox	Test Cypher queries against sample data	Optional but recommended
Redis	Key schema design and eviction policies	Yes (design only)
Prometheus / Grafana Cloud	Prototype dashboards and alerting rules	Yes (design); optional (prototype)
HashiCorp Vault	Key management architecture design	Yes (design only)

SECTION E5: AI-ASSISTED DEVELOPMENT POLICY

Interns are encouraged to use AI tools (Claude, ChatGPT, GitHub Copilot) for research, brainstorming, and drafting. However, all final deliverables must demonstrate the intern's own understanding. The following policy applies:

- PERMITTED: Using AI for researching fraud detection patterns, microservices best practices, technology comparisons, and generating initial drafts of documentation.
- PERMITTED: Using AI to generate sample configurations (YAML, JSON, Protobuf), Cypher queries, and Kubernetes manifests, which must then be reviewed and customised.
- NOT PERMITTED: Submitting AI-generated deliverables without substantial modification, critical evaluation, and personalisation.
- REQUIRED: Acknowledging AI-assisted content in a footnote or a dedicated /docs/AI_USAGE.md file.
- ENFORCEMENT: The Board evaluation will include probing questions to verify understanding. Inability to explain design decisions will result in significant point deductions.

SECTION E6: LEARNING RESOURCES

- Building Microservices (2nd Edition) by Sam Newman - O'Reilly. The definitive reference on microservices architecture, covering decomposition, communication, deployment, and testing.
- Designing Data-Intensive Applications by Martin Kleppmann - O'Reilly. Essential for understanding data consistency, event sourcing, stream processing, and distributed systems.
- Domain-Driven Design by Eric Evans - Addison-Wesley. The foundational text on bounded contexts and ubiquitous language.
- Kafka: The Definitive Guide (2nd Edition) by Gwen Shapira et al. - O'Reilly. Comprehensive Kafka architecture and design patterns.
- C4 Model Documentation by Simon Brown - https://c4model.com. The authoritative guide to C4 architectural diagramming.
- Istio in Action by Christian Posta and Rinor Maloku - Manning. Service mesh design with Istio and Envoy.
- YouTube: Martin Fowler on Microservices, Chris Richardson on Event-Driven Architecture, Confluent Kafka Tutorials, Neo4j Graph Academy.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- RBI Master Directions on Fraud Reporting: https://www.rbi.org.in. Required reading for understanding India-specific regulatory requirements.
- PCI DSS v4.0 Quick Reference Guide: https://www.pcisecuritystandards.org. Required for understanding payment card security requirements.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

END OF PROJECT DOCUMENT